

PROYECTO JAVA: PROGRAMACIÓN FUNCIONAL Y COLLECTIONS

Programación funcional:

¿Qué es? Es un paradigma de programación **declarativa** basado en el uso de funciones matemáticas. En la programación funcional las funciones son como "objetos". Es decir, se podría meter una función en una variable ya que el lenguaje no le da un trato especial o restrictivo, tiene los mismos derechos que un número o un texto.

Paradigma: Los paradigmas son variantes de los dos tipos principales de programación, imperativa y declarativa.

Un ejemplo de variantes de paradigmas de programación podría ser la programación orientada a objetos (POO) que vemos en clase, siendo esta una variante.

- **La programación imperativa:** es el más usado y más antiguo, se basa en dar instrucciones al ordenador de como hacer las cosas a través de algoritmos en lugar de describir el problema (como un libro de recetas) siendo cada paso como una instrucción.
- **La programación declarativa:** Se basa en describir el problema declarando propiedades y reglas que deben cumplirse. La solución se obtiene mediante mecanismos internos de control sin especificar cómo encontrarla, solo se le tiene que indicar al ordenador que estamos buscando. En la programación declarativa existe la inmutabilidad, es decir, que una vez que se le asigne un valor a una variable este lo mantendrá para siempre (no podemos "machacar" el valor de una variable), las variables también son usadas con transparencia referencial (lo que significa que una expresión puede ser sustituida por su resultado sin que nada cambie en el programa.) Esto permite el uso de mecanismos matemáticos para optimizar el rendimiento de los programas.
- **Programación funcional:** En matemáticas, un predicado es una expresión que afirma algo sobre un sujeto y que solo puede ser

verdadero o falso. En programación funcional se usan constantemente estos predicados para tomar decisiones sobre los datos. Es una programación de corte más matemático (Python por ejemplo también representa parcialmente este paradigma).

La programación funcional se basa en los cálculos lambda, sistema desarrollado en 1930 diseñado para investigar la naturaleza de las funciones, computabilidad y su relación con la recursión (ya profundizará nuestro compañero Paco). Los lenguajes funcionales priorizan la aplicación de funciones de orden superior para resolver problemas que en otros lenguajes se resolverían mediante estructuras de control. En programación funcional pura, si dos expresiones son sintácticamente iguales entonces siempre devolverán el mismo resultado. Es decir, son lenguajes con transparencia referencial. (aprovechado para diseñar estrategias que eviten repetir el cómputo de expresiones antes vistas). Los lenguajes de uso específico usados comúnmente como SQL utilizan algunos elementos de programación funcional.

Conceptos Técnicos.

Para asegurar la transparencia referencial se necesitan objetos inmutables, pero también funciones que siempre devuelvan el mismo valor ante las mismas entradas. En la programación funcional, queremos que las funciones sean trozos de código **independientes y predecibles**. Para lograrlo, hay dos reglas:

1. Evitar los Efectos Secundarios

Un efecto secundario ocurre cuando una función además de calcular un resultado modifica algo que está fuera de ella.

- **Evitamos:** Que una función cambie una variable global, modifique un archivo o altere una base de datos.
- **Objetivo:** Que la función no deje rastro y que no altere nada mientras se ejecuta.

2. Crear "Funciones Puras"

Una función se considera **pura** cuando cumple estas dos condiciones:

C/ CONDES DE BUSTILLO 17 | 41010 SEVILLA | T 954 331 488 | <https:// triana.salesianos.edu>

- **Previsibilidad:** Si metes los mismos datos siempre te devolverá el mismo resultado .
- **Aislamiento:** No cambia ningún elemento. Recibe información, la procesa y devuelve un valor nuevo.

Manejo de funciones.

Funciones de primera clase: Se les llama funciones de primera clase cuando se pueden almacenar en variables, pasar como parámetro y devolver desde funciones, sin ningún trato especial.

Funciones de orden superior: Cuando una función recibe otras funciones como parámetro, se la denomina de orden superior.

Las más conocidas son **map** y **reduce**: la primera aplica la misma función sobre cada elemento de una colección, y la segunda acumula los elementos en un único valor según una función dada.

Recursión: Es una forma de resolver problemas haciendo que una función se llame a sí misma (como si fueran bucles), pero cada vez con una versión más simple del problema hasta llegar a un caso sencillo que se puede resolver directamente (caso base). Es una alternativa a los mencionados bucles, ya que en esta programación no se suelen usar estructuras for o while, en lugar de repetir código con un bucle, la repetición se consigue mediante llamadas recursivas a la función.

Operaciones Esenciales.

Map: El uso de map aplica una determinada función a todos los elementos de una entrada o lista. Map es normalmente usado con funciones lambda.

Forma:

map(funcion_a_aplicar, lista de entradas)

Filter: Crea una lista de elementos que si se usan en la llamada de una función devuelve true (filtra los elementos de una lista usando un criterio determinado), esta función es similar al bucle.

Reduce: Es una función útil cuando queremos realizar operaciones sobre una lista y que nos devuelvan su resultado (por ejemplo calcular el producto de todos los elementos y devolverlo en un único valor).

COLLECTIONS

¿Qué son?

Son estructuras de datos que permiten guardar muchos elementos en una sola variable, haciendo más fácil operaciones como añadir, eliminar, buscar o recorrer elementos.

A diferencia de los arrays las collections son **dinámicas** (pueden cambiar de tamaño automáticamente).

Interfaces principales

- List → Colecciones ordenadas que permiten elementos repetidos
- Set → Colecciones que no permiten duplicados y sin orden
- Queue → Colecciones diseñadas para manejar elementos en orden

Implementaciones más usadas

ArrayList (implementa List)

→ permite acceso rápido por índice

LinkedList (implementa List y Queue)

→ Mejor para insertar o eliminar elementos

HashSet (implementa Set)

→ No permite duplicados y no tiene orden

HashMap (aunque es Map, tiene relación)

→ Guarda datos en formato clave-valor (key, value)

Ventajas

- Tamaño dinámico
- Gran variedad de estructuras según necesidad

- Integración con programación funcional (Streams, lambdas)

Desventajas

-

Métodos:

1 Disjoint

```
static boolean disjoint(Collection<?> c1, Collection<?> c2){  
  
}
```

Este método nos devuelve un booleano true si al pasarle dos colecciones y compararlas no tienen ningún elemento en común, sino, devuelve false.

2 ReverseOrder

```
static <T> Comparator<T> reverseOrder(){  
  
}
```

Este método devuelve un comparador que impone el orden inverso al natural en una colección de objetos que implementan la interfaz Comparable.

3 Unmodifiable

```
static <T> Collection<T> unmodifiableCollection(Collection<? extends T> c){  
  
}
```

Este método sirve para crear una vista de solo lectura de una colección específica.

Tiene variantes como unmodifiableList, Set, Map, SortedSet, SortedMap...

4 Synchronized

```
static <T> Collection<T> synchronizedCollection(Collection<T> c) {  
  
}
```

Este método mete una colección no segura (como ArrayList o HashMap) en un contenedor que tiene sincronización de hilos, asegurándose de que solo un hilo pueda modificar o leer la colección a la vez para evitar la "corrupción" de datos en entornos

5 BinarySearch

```
static <T> int binarySearch(List<? extends T> list, T key, Comparator<? super T> c)
```

Este método busca algún objeto especificado en la lista usando el algoritmo binario

6 Frequency

```
static int frequency(Collection<?> c, Object o){  
}
```

Este método se usa para saber cuántas veces exactas se repite un objeto en una colección determinada.

EJEMPLO:

```
package ejercicio_tienda;
```

```
public class Videojuego {  
    private String codigo;  
    private String titulo;  
    private double precio;  
    private int stock;  
  
    public String getCodigo() {  
        return codigo;  
    }  
  
    public void setCodigo(String codigo) {
```

```
        this.codigo = codigo;
    }

    public String getTitulo() {
        return titulo;
    }

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public int getStock() {
        return stock;
    }

    public void setStock(int stock) {
        this.stock = stock;
    }

    public Videojuego(String codigo, String titulo, double precio, int stock) {
        super();
        this.codigo = codigo;
        this.titulo = titulo;
        this.precio = precio;
        this.stock = stock;
    }
}
```

[@Override](#)

```
        public String toString() {  
            return "Videojuego [codigo=" + codigo + ", titulo=" + titulo + ", precio=" +  
precio + ", stock=" + stock + "];"  
        }  
    }  
}  
  
package ejercicio_tienda;  
  
import java.util.HashMap;  
import java.util.Map;  
  
public class Tienda {  
    private Map<String, Videojuego> listaJuegos = new HashMap<>();  
  
    public Tienda(Map<String, Videojuego> listaJuegos) {  
        super();  
        this.listaJuegos = listaJuegos;  
    }  
  
    public void agregar(String codigo, Videojuego v) {  
        listaJuegos.put(codigo, v); // Método put del API [cite: 127]  
    }  
  
    public Videojuego buscarPorCodigo(String codigo) {  
        for (Map.Entry<String, Videojuego> entry : listaJuegos.entrySet()) {  
            if (entry.getKey().equalsIgnoreCase(codigo)) {  
                return entry.getValue();  
            }  
        }  
        return null;  
    }  
}
```

```
public Videojuego buscarDirecto(String codigo) {  
    return listaJuegos.get(codigo);
```

```
}
```

```
public void borrarJuego(String codigo) {
```

```
    listaJuegos.remove(codigo);
```

```
}
```

```
public void mostrarTodo() {
```

```
    for (Map.Entry<String, Videojuego> entry : listaJuegos.entrySet()) {
```

```
        System.out.println(entry.getValue()); // [cite: 159]
```

```
    }
```

```
}
```

```
public double calcularInversionTotal() {
```

```
    double suma = 0;
```

```
    for (Videojuego v : listaJuegos.values()) {
```

```
        suma += (v.getPrecio() * v.getStock());
```

```
    }
```

```
    return suma;
```

```
}
```

```
}
```

```
package ejercicio_tienda;
```

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
import utilidades.Leer;
```

```
public class Principal {  
    public static void main(String[] args) {  
        int elegir, stock;  
  
        String codigo, titulo;  
        double precio;  
  
        Map<String, Videojuego> lista = new HashMap<>();  
        Tienda t = new Tienda(lista);  
  
        do {  
            System.out.println("1. Añadir Videojuego");  
            System.out.println("2. Borrar por código");  
            System.out.println("3. Mostrar catálogo");  
            System.out.println("4. Calcular inversión total");  
            System.out.println("0. Salir");  
  
            elegir = Leer.datoInt();  
  
            switch (elegir) {  
                case 1:  
                    System.out.println("Dime el código:");  
                    codigo = Leer.dato();  
                    System.out.println("Dime el título:");  
                    titulo = Leer.dato();  
                    System.out.println("Dime el precio:");  
                    precio = Leer.datoDouble();  
                    Svstem.out.println("Dime el stock:");
```



```
stock = Leer.datoInt();

Videojuego v = new Videojuego(codigo, titulo, precio,
stock);

t.agregar(codigo, v);
break;

case 2:
    System.out.println("Dime el código del juego a borrar:");
    codigo = Leer.dato();
    t.borrarJuego(codigo);
    break;

case 3:
    t.mostrarTodo();
    break;

case 4:
    System.out.printf("El valor total es: %.2f€\n",
t.calcularInversionTotal());
    break;
    }
    } while (elegir != 0);
    }
}
```

Bibliografía:

<https://www.campusmvp.es/recursos/post/programacion-funcional-inmutabilidad-y-funciones-puras.aspx>

<https://verso.mat.uam.es/~pablo.angulo/doc/laboratorio/b1s3.html>

https://es.wikipedia.org/wiki/Programaci%C3%B3n_funcional

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Collections.html>